TASK 1.1 : JAVA LAMBDA EXPRESSION

PROGRAM:

```
import java.io.*;
import java.util.*;
interface PerformOperation {
 boolean check(int a);
}
class MyMath {
 public static boolean checker(PerformOperation p, int num) {
  return p.check(num);
 }

public static PerformOperation isOdd() {
    return n -> n % 2 != 0;
}
public static PerformOperation isPrime() {
    return n -> {
        if (n < 2) return false;

        for (int i = 2; i < n; i++) {
            if (n % i == 0) return false;
        }
        return true;
    };
}
public static PerformOperation isPalindrome() {
    return n -> {
        int original = n;
        int reverse = 0
        while (n > 0) {
```

```
            reverse = reverse * 10 + n % 10;
            n = n / 10;
        }
        return original == reverse;
    };
}
}
```

OUTPUT:

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

TASK 1.2:SALARY TRANSFORMATION SYSTEM

PROGRAM:

```
import java.util.*;
import java.util.stream.*;


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Integer> salaries = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            salaries.add(sc.nextInt());
        }
        salaries.stream()
                .map(salary -> salary + (salary * 10 / 100))
                .forEach(salary -> System.out.print(salary + " "));
    }
}
```

OUTPUT:

```
5
1000 2000 3000 4000 5000
1100 2200 3300 4400 5500
```

TASK 1.3:JAVA SORT

PROGRAM:

```
import java.util.*;
class Student{
    private int id;
    private String fname;
    private double cgpa;
    public Student(int id, String fname, double cgpa) {
        super();
        this.id = id;
        this.fname = fname;
        this.cgpa = cgpa;
    }
    public int getId() {
        return id;
    }
    public String getFname() {
        return fname;
    }
    public double getCgpa() {
        return cgpa;
    }
}
public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());
        List<Student> studentList = new ArrayList<Student>();
        while(testCases>0){
```

```
            int id = in.nextInt();
            String fname = in.next();
            double cgpa = in.nextDouble();
            Student st = new Student(id, fname, cgpa);
            studentList.add(st);
            testCases--;
        }
        Collections.sort(studentList, new Comparator<Student>() {
            @Override
            public int compare(Student s1, Student s2) {

                if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
                    return Double.compare(s2.getCgpa(), s1.getCgpa());
                }
                // If CGPA is same, compare by First Name (Ascending)
                else if (!s1.getFname().equals(s2.getFname())) {
                    return s1.getFname().compareTo(s2.getFname());
                }
                // If CGPA and First Name are same, compare by ID (Ascending)
                else {
                    return Integer.compare(s1.getId(), s2.getId());
                }
            }
        });
          for(Student st: studentList){
            System.out.println(st.getFname());
        }
    }
}
```

OUTPUT:

Your Output (stdout)

```
1 Ashis
2 Fahim
3 Samara
4 Samiha
5 Rumpa
```

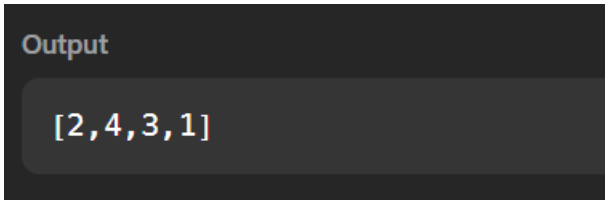
TASK 1.4:SORT ARRAY BY PARITY

PROGRAM:

```
class Solution {
    public int[] sortArrayByParity(int[] nums) {
        return java.util.stream.IntStream.concat(
                Arrays.stream(nums).filter(n -> n % 2 == 0),
                Arrays.stream(nums).filter(n -> n % 2 != 0)
        ).toArray();

    }
}
```

OUTPUT:



TASK 1.5:SORTING COMPARATOR

PROGRAM:

```
import java.util.*;
class Player {
    String name;
    int score;
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}
class Checker implements Comparator<Player>
     public int compare(Player a, Player b) {
        return Comparator
                .comparingInt((Player p) -> p.score)
                .reversed()
                .thenComparing(p -> p.name)
                .compare(a, b)
    }
}
```

OUTPUT:

Your Output (stdout)

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

TASK 1.6:JAVA COMPARATOR

PROGRAM:

```
import java.util.*;
class Checker implements Comparator<Player> {
   public int compare(Player a, Player b){
      return Comparator
            .comparingInt((Player p)-> p.score)
            .reversed()
            .thenComparing(p -> p.name)
            .compare(a,b);
   }
}
```

OUTPUT:

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

TASK 1.7:RICHEST CUSTOMER WEALTH

PROGRAM:

```
import java.util.Arrays;
class Solution {
    public int maximumWealth(int[][] accounts) {
        return Arrays.stream(accounts)
                .mapToInt(customer -> Arrays.stream(customer).sum())
                .max()
                .getAsInt();
    }
}
```

OUTPUT:



WEEK – 2

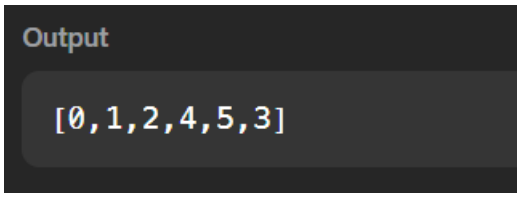
TASK 2.1: BUILD ARRAY FROM PERMUTATION

PROGRAM:

```
import java.util.*;
import java.util.function.*;


class Solution {
    public int[] buildArray(int[] nums) {
        return IntStream.range(0, nums.length)
                .map(i -> nums[nums[i]])
                .toArray();
    }
}
```

OUTPUT:



TASK 2.2:SHUFFLE THE ARRAY

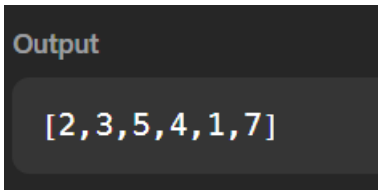
PROGRAM:

```
import java.util.*;
import java.util.stream.*;


class Solution {
    public int[] shuffle(int[] nums, int n) {
        return IntStream.range(0,n)
                .flatMap(i-> IntStream.of(nums[i], nums[i+n]))
                .toArray();


    }
}
```

OUTPUT:

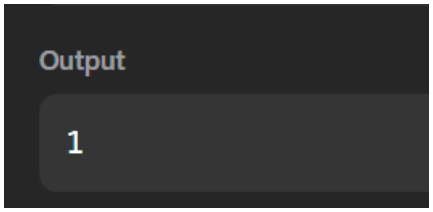


TASK 2.3 : FINDING THE HIGHEST ALTITUDE

PROGRAM:

```
class Solution {
    public int largestAltitude(int[] gain) {
        int current = 0;
        int highest = 0;
        for (int i =0; i < gain.length; i++) {
            current = current + gain[i];
            if (current > highest) {
                highest = current;
            }
        }
        return highest;
    }
}
```

OUTPUT:



TASK 2.4: JAVA DEQUEUE

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int m = sc.nextInt();
      Deque<Integer> deque = new ArrayDeque<>();
      HashSet<Integer> set = new HashSet<>();
      int maxUnique = 0;
      for (int i = 0; i < n; i++) {
         int value = sc.nextInt();
         deque.addLast(value);
         set.add(value);
         if (deque.size() == m) {
            maxUnique = Math.max(maxUnique, set.size());
            int first = deque.removeFirst();
            if (!deque.contains(first)) {
               set.remove(first);
            }
         }
      }
      System.out.println(maxUnique);
      sc.close();
   }
}
```

OUTPUT:

Your Output (stdout)

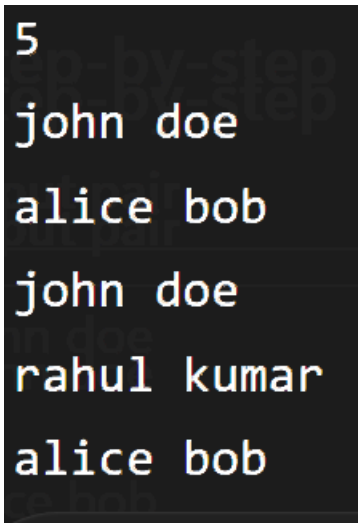
```
1  3
```

TASK 2.5:JAVA HASHSET

PROGRAM:

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        HashSet<String> set = new HashSet<>();
        for (int i = 0; i < n; i++) {
            String first = sc.next();
            String second = sc.next();
            String pair = first + " " + second;
            set.add(pair);
            System.out.println(set.size());
        }
        sc.close();
    }
}
```

OUTPUT:



WEEK 3

TASK 3.1 : JAVA DATE AND TIME

PROGRAM:

```
class Result {


    public static String findDay(int month, int day, int year) {
        java.util.Calendar cal = java.util.Calendar.getInstance
        cal.set(year, month - 1,day);
        return cal.getDisplayName(
            java.util.Calendar.DAY_OF_WEEK,
            java.util.Calendar.LONG,
            Locale.ENGLISH
        ).toUpperCase();


    }


}
```

OUTPUT:

Input (stdin)

```
08 05 2015
```

Expected Output

```
WEDNESDAY
```

TASK 3.2: NUMBER OF DAYS BETWEEN TWO DAYS

PROGRAM:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

class Solution {
    public int daysBetweenDates(String date1, String date2) {

        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);
        return (int) Math. Abs(ChronoUnit.DAYS.between(d1,d2));
    }
}
```

OUTPUT:



TASK 3.3:DAY OF THE YEAR

PROGRAM:

```
import java.time.LocalDate;

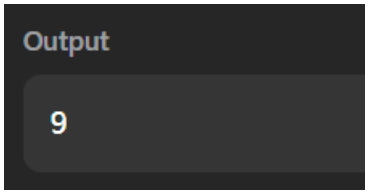
class Solution {
    public int dayOfYear(String date) {

        LocalDate d = LocalDate.parse(date);

        return d.getDayOfYear();

    }
}
```

OUTPUT:



TASK 3.4: DAY OF THE WEEK

PROGRAM:

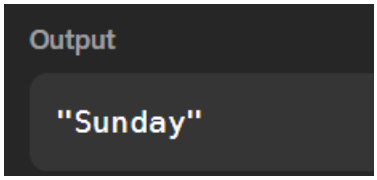
```
import java.time.LocalDate;
import java.time.format.TextStyle;
import java.util.Locale;


class Solution {
    public String dayOfTheWeek(int day, int month, int year) {


        LocalDate date = LocalDate.of(year,month, day);


        return date.getDayOfWeek()
                .getDisplayName(TextStyle.FULL, Locale.ENGLISH);
    }
}
```

OUTPUT:



TASK 3.5: JAVA PRIORITY QUEUE

PROGRAM:

```
import java.util.*;
class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    public int getID() {
        return id;
    }
    public String getName() {
        return name;
    }
    public double getCGPA() {
        return cgpa;
    }
}
class Priorities {
    public List<Student> getStudents(List<String> events) {
        PriorityQueue<Student> pq = new PriorityQueue<>(
            (s1, s2) ->
                if (s1.getCGPA() != s2.getCGPA()) {
                    return Double.compare(
                        s2.getCGPA(),
                        s1.getCGPA()
```

```
            );
        }
        int nameCompare =
            s1. getName().compareTo(s2.getName());
        if (nameCompare != 0) {
            return nameCompare;
        }
        return Integer.compare(
            s1.getID(),
            s2.getID()
        );
    }
);
for (String event : events) {
    if (event.startsWith("ENTER")) {
        String[] parts = event.split(" ");
        String name = parts[1];
        double cgpa = Double.parseDouble(parts[2]);
        int id = Integer.parseInt(parts[3]);
        pq.add(new Student(id, name, cgpa));
    } else if (event.equals("SERVED")) {
        if (!pq.isEmpty()) {
            pq.poll();
        }
    }
}
List<Student> result = new ArrayList<>();
while (!pq.isEmpty()) {
    result.add(pq.poll());
}
```

```
        return result;
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();
        List<String> events = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            events.add(sc.nextLine());
        }
        Priorities priorities = new Priorities();
        List<Student> students =
            priorities.getStudents(events);
        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student student : students) {
                System.out.println(student.getName());
            }
        }

        sc.close();
    }
}
```

OUTPUT:

Expected Output

```
Dan
Ashley
Shafaet
Maria
```

TASK 3.6:JAVA ARRAYLIST

Program:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<ArrayList<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();
            ArrayList<Integer> numbers = new ArrayList<>();
            for (int j = 0; j < d; j++) {
                numbers.add(sc.nextInt());
            }
            list.add(numbers);
        }

        int q = sc.nextInt();
        for (int i = 0; i < q; i++) {
            int x = sc.nextInt();
            int y = sc.nextInt();
            try {
                System.out.println(list.get(x - 1).get(y - 1));
            }
            catch (IndexOutOfBoundsException e) {
                System.out.println("ERROR!");
            }
        }
```

```
        sc.close();
    }
}
```

OUTPUT:

Expected Output

```
74
52
37
ERROR!
ERROR!
```

TASK 3.7:LARGEST NUMBER

PROGRAM:

```
class Solution {
    public String largestNumber(int[] nums) {
        String[] arr= new String[nums.length];
        for (int i = 0;i < nums.length; i++) {
            arr[i] = nums[i] + "";
        }
        Arrays.sort(arr, (a,b) -> (b + a).compareTo(a+b));
        if (arr[0].equals("0")) {
            return "0";
        }
        String answer = "";
        for (String s: arr) {
            answer = answer + s;
        }
        return answer;

    }
}
```

OUTPUT:



TASK 3.8:JAVA COMPARATOR

PROGRAM:

```
import java.util.*;
class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
        return a.name.compareTo(b.name);
    }
}
class Player{
    String name;
    int score;
    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}
class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
```

```
        Arrays.sort(player, checker);
        for(int i = 0; i < player. Length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

OUTPUT:

Expected Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

TASK 3.9:SORT THE PEOPLE

PROGRAM:

```
import java.util.*;
class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        Integer[] index = new Integer[names.length];
        for (int i = 0; i < names.length; i++) {
            index[i] = i;
        }
        Arrays.sort(index, (a, b) -> heights[b] - heights[a]);
        String[] result = new String[names.length];
        for (int i = 0; i < names.length; i++) {
            result[i] = names[index[i]];
        }
        return result;
    }
}
```

OUTPUT:



WEEK – 4

TASK 4.1: DETERMINE-IF-STRING-HALVES-ARE-LIKE

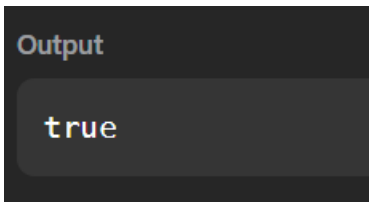
PROGRAM:

```
class Solution {
    public boolean halvesAreAlike(String s) {
        int mid = s.length() / 2;
        long first = s.substring(0, mid)
                .chars()
                .filter(c -> "aeiouAEIOU".indexOf(c) != -1)
                .count();

        long second = s.substring(mid)
                .chars()
                .filter(c -> "aeiouAEIOU".indexOf(c) != -1)
                .count();

        return first == second;
    }
}
```

OUTPUT:



TASK 4.2 : LAPIN

PROGRAM:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            String s = sc.next();
            int n = s.length();
            int mid = n / 2;
            int[] first = new int[26];
            int[] second = new int[26];
            for (int i = 0; i < mid; i++) {
                first[s.charAt(i) - 'a']++;
            }
            for (int i = (n + 1) / 2; i < n; i++) {
                second[s.charAt(i) - 'a']++;
            }
            if (Arrays.equals(first, second)) {
                System.out.println("YES");
            } else {
                System.out.println("NO");
            }
        }
        sc.close();
    }
}
```

OUTPUT:



TASK 4.3 : COMPARE THE TRIPLETS

PROGRAM:

```
import java.util.*;
class Result {
    public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) {
        int aliceScore = 0;
        int bobScore = 0;
        for (int i = 0; i < 3; i++) {
            if (a.get(i) > b.get(i)) {
                aliceScore++;
            }
            else if (a.get(i) < b.get(i)) {
                bobScore++;
            }
        }
        return Arrays.asList(aliceScore, bobScore);
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Integer> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            a.add(sc.nextInt());
        }
        for (int i = 0; i < 3; i++) {
            b.add(sc.nextInt());
        }
```

```
        List<Integer> result = Result.compareTriplets(a, b);
        for (int score : result) {
            System.out.print(score + " ");
        }
        sc.close();
    }
}
```

OUTPUT:

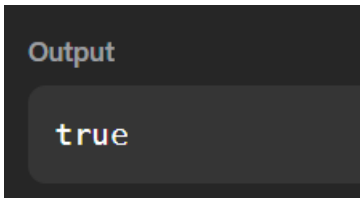


TASK 4.4 : CONTAINS DUPLICATES

PROGRAM:

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();
        for (int n : nums) {
            if(set.contains(n)) {
                return true;
            }
            set.add(n);
        }
        return false;
    }
}
```

OUTPUT:



TASK 4.5 : TIME CONVERSION

PROGRAM:

```
import java.io.*;
import java.util.*;
class Result {
    public static String timeConversion(String s) {
        String period = s.substring(8, 10);
        int hour = Integer.parseInt(s.substring(0, 2));
        if (period.equals("AM")) {
            if (hour == 12) {
                hour = 0;
            }
        }
        else {
            if (hour != 12) {
                hour = hour + 12;
            }
        }
        return String.format("%02d", hour) + s.substring(2, 8);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter =
            new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
        String s = bufferedReader.readLine();
        String result = Result.timeConversion(s);
```

```
        bufferedWriter.write(result);
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Expected Output

```
1   19:05:45
```

TASK 4.6 :MOVE ZEROES

PROGRAM:

```
class Solution {
    public void moveZeroes(int[] nums) {

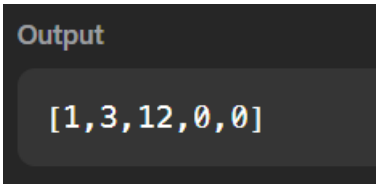
        int j = 0;

        for (int i = 0; i < nums.length; i++) {

            if (nums[i] != 0) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;

                j++;
            }
        }
    }
}
```

OUTPUT:



TASK 4.7:DIAGONAL DIFFERENCE

PROGRAM:

```
import java.util.*;

public class Solution {

    public static int diagonalDifference(List<List<Integer>> arr) {
        int n = arr.size();
        int leftDiagonal = 0;
        int rightDiagonal = 0;

        for (int i = 0; i < n; i++) {
            leftDiagonal += arr.get(i).get(i);
            rightDiagonal += arr.get(i).get(n - 1 - i);
        }

        return Math.abs(leftDiagonal - rightDiagonal);
    }
}
```

OUTPUT:



TASK 4.8: TRANSPOSE MATRIX

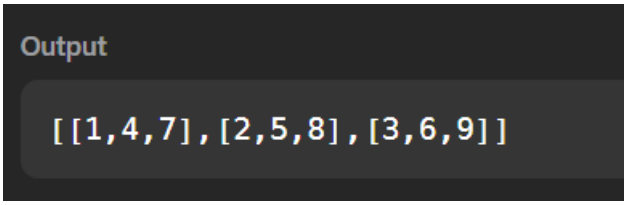
PROGRAM:

```
import java.util.stream.IntStream;

class Solution {
    public int[][] transpose(int[][] matrix) {
        int rows = matrix.length;
        int cols = matrix[0].length;

        return IntStream.range(0, cols)
                .mapToObj(j ->
                        IntStream.range(0,rows)
                                .map(i -> matrix[i][j])
                                .toArray()
                )
                .toArray(int[][]::new);
    }
}
```

OUTPUT:



TASK 4.9:MATRIX BLOCK SUM

PROGRAM:

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {

        int m = mat.length;
        int n = mat[0].length;

        int[][] answer = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {

                int sum = 0;

                for (int r = Math.max(0, i - k);
                     r <= Math.min(m - 1, i + k);
                     r++) {

                    for (int c = Math.max(0, j - k);
                         c <= Math.min(n - 1, j + k);
                         c++) {

                        sum += mat[r][c];
                    }
                }

                answer[i][j] = sum;
            }
        }

        return answer;
    }
}
```

OUTPUT:



TASK 4.10: MATRIX ROTATION

PROGRAM:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void matrixRotation(List<List<Integer>> matrix, int r) {

        int rows = matrix.size();
        int cols = matrix.get(0).size();

        int[][] result = new int[rows][cols];

        for (int layer = 0; layer < Math.min(rows, cols) / 2; layer++) {

            List<Integer> elements = new ArrayList<>();

            for (int j = layer; j < cols - layer; j++) {
                elements.add(matrix.get(layer).get(j));
            }

            for (int i = layer + 1; i < rows - layer; i++) {
                elements.add(matrix.get(i).get(cols - layer - 1));
            }
            for (int j = cols - layer - 2; j >= layer; j--) {
                elements.add(matrix.get(rows - layer - 1).get(j));
            }


            for (int i = rows - layer - 2; i > layer; i--) {
                elements.add(matrix.get(i).get(layer));
            }

            int size = elements.size();
            int rotation = r % size;
            int index = 0;
            for (int j = layer; j < cols - layer; j++) {
                result[layer][j] =
                    elements.get((index + rotation) % size);
                index++;
            }
            for (int i = layer + 1; i < rows - layer; i++) {
                result[i][cols - layer - 1] =
                    elements.get((index + rotation) % size);
                index++;
            }
            for (int j = cols - layer - 2; j >= layer; j--) {
                result[rows - layer - 1][j] =
                    elements.get((index + rotation) % size);
                index++;
            }
```

```
            for (int i = rows - layer - 2; i > layer; i--) {
                result[i][layer] =
                    elements.get((index + rotation) % size);
                index++;
            }
        }

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(result[i][j]);

                if (j < cols - 1) {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }

    public static void main(String[] args) throws IOException {

        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        String[] firstLine = br.readLine().trim().split(" ");

        int rows = Integer.parseInt(firstLine[0]);
        int cols = Integer.parseInt(firstLine[1]);
        int r = Integer.parseInt(firstLine[2]);

        List<List<Integer>> matrix = new ArrayList<>();

        for (int i = 0; i < rows; i++) {

            String[] values = br.readLine().trim().split(" ");

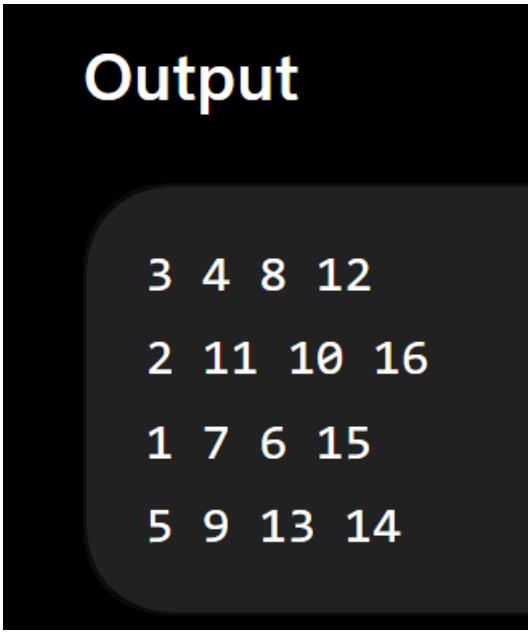
            List<Integer> row = new ArrayList<>();

            for (String value : values) {
                row.add(Integer.parseInt(value));
            }

            matrix.add(row);
        }

        matrixRotation(matrix, r);
    }
}
```

OUTPUT:



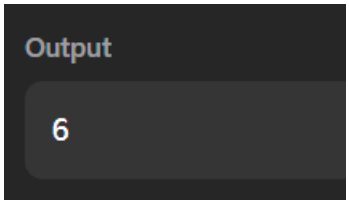
WEEK 5

TASK 5.1 : MAXIMUM SUBARRAY

PROGRAM:

```
class Solution {
    public int maxSubArray(int[] nums) {
        int currentSum = nums[0];
        int maxSum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum;
    }
}
```

OUTPUT:



TASK 5.2: THE BIRTHDAY BAR

PROGRAM:

```
import java.util.*;
public class Solution {
    public static int birthday(List<Integer> s, int d, int m) {
        int count = 0;
        for (int i = 0; i <= s.size() - m; i++) {
            int sum = 0;
            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }
            if (sum == d) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            s.add(sc.nextInt());
        }
        int d = sc.nextInt();
        int m = sc.nextInt();
        System.out.println(birthday(s, d, m));
        sc.close();
    }
}
```

OUTPUT:



TASK 5.3: MAXSUBARRAY

PROGRAM:

```
import java.io.*;
import java.util.*;
class Result {
    public static List<Integer> maxSubarray(List<Integer> arr) {
        int current = arr.get(0);
        int maxSubarray = arr.get(0);
        int maxSubsequence = 0;
        int largestNegative = arr.get(0);
        for (int num : arr) {
            if (num != arr.get(0)) {
                current = Math.max(num, current + num);
                maxSubarray = Math.max(maxSubarray, current);
            }
            if (num > 0) {
                maxSubsequence += num;
            }
            if (num > largestNegative) {
                largestNegative = num;
            }
        }
        if (maxSubsequence == 0) {
            maxSubsequence = largestNegative;
        }
        List<Integer> result = new ArrayList<>();
        result.add(maxSubarray);
        result.add(maxSubsequence);
        return result;
    }
```

```
}
public class Solution {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            int n = Integer.parseInt(br.readLine().trim());
            List<Integer> arr = new ArrayList<>();
            while (arr.size() < n) {
                String[] parts = br.readLine().trim().split("\\s+");
                for (String s : parts) {
                    if (!s.isEmpty()) {
                        arr.add(Integer.parseInt(s));
                    }
                }
            }
            List<Integer> result = Result.maxSubarray(arr);
            System.out.println(result.get(0) + " " + result.get(1));
        }
    }
}
```

OUTPUT:

Expected Output

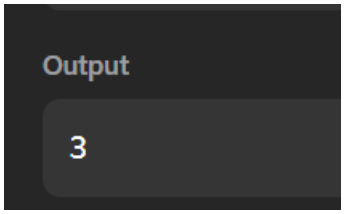
```
1 1
-1 -1
1 1
6 6
-10 -10
5 6
```

TASK 5.4:MAXIMUM SUB CIRCULAR SUBARRAY

PROBLEM:

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0;
        int currentMax = 0;
        int maxSum = nums[0];
        int currentMin = 0;
        int minSum = nums[0];
        for (int num : nums) {
            currentMax = Math.max(num, currentMax + num);
            maxSum = Math.max(maxSum, currentMax);
            currentMin = Math.min(num, currentMin + num);
            minSum = Math.min(minSum, currentMin);
            total += num;
        }
        if (maxSum < 0) {
            return maxSum;
        }
        return Math.max(maxSum, total - minSum);
    }
}
```

OUTPUT:

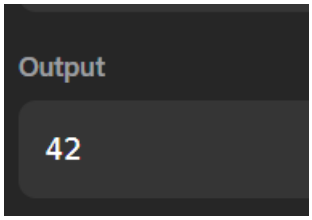


TASK 5.5:STRING TO INTEGER

PROGRAM:

```
class Solution {
    public int myAtoi(String s) {
        int i = 0;
        int n = s.length();
        while (i < n && s.charAt(i) == ' ') {
            i++;
        }
        int sign = 1;
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            if (s.charAt(i) == '-') {
                sign = -1;
            }
            i++;
        }
        long num = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            num = num * 10 + (s.charAt(i) - '0');
            if (sign == 1 && num > Integer.MAX_VALUE) {
                return Integer.MAX_VALUE;
            }
            if (sign == -1 && -num < Integer.MIN_VALUE) {
                return Integer.MIN_VALUE;
            }
            i++;
        }
        return (int)(sign * num);
    }
}
```

OUTPUT:



TASK 5.6 : ALTERNATING CHARACTERS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static int alternatingCharacters(String s) {
        int deletions = 0;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                deletions++;
            }
        }
        return deletions;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in)
        );
        int q = Integer.parseInt(br.readLine().trim());
        for (int i = 0; i < q; i++) {
            String s = br.readLine().trim();
            int result = alternatingCharacters(s);
            System.out.println(result);
        }
        br.close();
    }
}
```

OUTPUT:

Expected Output

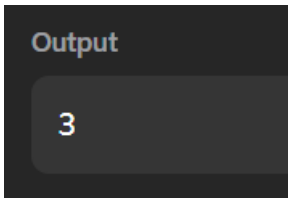
```
3
4
0
0
4
```

TASK 5.7 :LONGEST SUBSTRING

PROGRAM:

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int left = 0;
        int maxLength = 0;
        HashSet<Character> set = new HashSet<>();
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

OUTPUT:



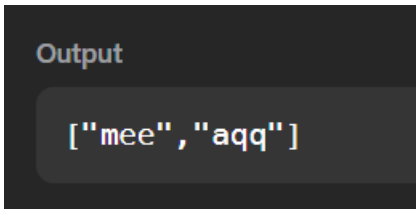
TASK 5.8:FIND AND REPLACE PATTERN

PROGRAM:

```
import java.util.*;
class Solution {
    public List<String> findAndReplacePattern(String[] words, String pattern) {
        List<String> result = new ArrayList<>();
        for (String word : words) {
            if (matches(word, pattern)) {
                result.add(word);
            }
        }
        return result;
    }
    private boolean matches(String word, String pattern) {
        Map<Character, Character> map = new HashMap<>();
        Map<Character, Character> reverseMap = new HashMap<>();
        for (int i = 0; i < word.length(); i++) {
            char w = word.charAt(i);
            char p = pattern.charAt(i);
            if (map.containsKey(p) && map.get(p) != w) {
                return false;
            }
            if (reverseMap.containsKey(w) && reverseMap.get(w) != p) {
                return false;
            }
            map.put(p, w);
            reverseMap.put(w, p);
        }
        return true;
    }
```

}

OUTPUT:

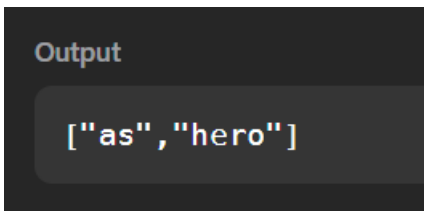


TASK 5.9: STRING MATCHING IN AN ARRAY

PROGRAM:

```
import java.util.*;
class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        for (int i = 0; i < words.length; i++) {
            for (int j = 0; j < words.length; j++) {
                if (i != j && words[j].contains(words[i])) {
                    result.add(words[i]);
                    break;
                }
            }
        }
        return result;
    }
}
```

OUTPUT:



TASK 5.10 :NAÏVE ALGORITHM

PROGRAM:

```
import java.util.ArrayList;
public class GFG {
    static ArrayList<Integer> search (String pat, String txt) {
        int m = pat.length();
        int n = txt.length();
        ArrayList<Integer> ans = new ArrayList<Integer>();
        for (int i = 0; i <= n - m; i++) {
            int j;
            for (j = 0; j < m; j++) {
                if (txt.charAt(i + j) != pat.charAt(j)) {
                    break;
                }
            }
            if (j == m) {
                ans.add(i);
            }
        }
        return ans;
    }
    public static void main(String[] args) {
        String txt = "aabaacaadaabaaba";
        String pat = "aaba";
        ArrayList<Integer> res = search(pat, txt);
        for (int i = 0; i < res.size(); i++) {
            System.out.print(res.get(i));
            if (i < res.size() - 1) {
                System.out.print(" ");
            }
```

```
        }
    }
}
```

OUTPUT:

Output

```
0 9 12
```

WEEK 6

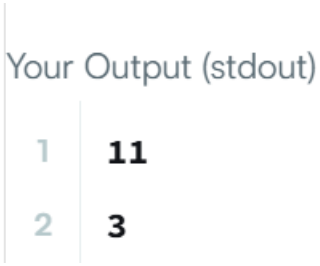
TASK 6.1: STRING SIMILARITY

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static long stringSimilarity(String s) {
        int n = s.length();
        int[] z = new int[n];
        int left = 0;
        int right = 0;
        long sum = n;
        for (int i = 1; i < n; i++) {
            if (i <= right) {
                z[i] = Math.min(right - i + 1, z [i - left]);
            }
            while (i + z[i] < n &&
                   s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }
            if (i + z[i] - 1 > right) {
                left = i;
                right = i + z[i] - 1;
            }
            sum += z[i];
        }
        return sum;
    }
    public static void main(String[] args) throws Exception {
      BufferedReader br =
```

```
            new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            String s = br.readLine().trim();
            System.out.println(stringSimilarity(s));
        }
    }
}
```

OUTPUT:



TASK 6.2:REPEATED SUBSTRING

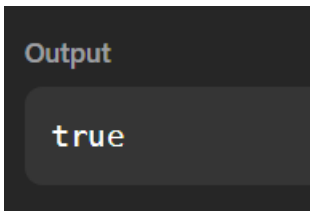
PROGRAM:

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {

        String temp = s + s;

        return temp.substring(1, temp.length() - 1).contains(s);
    }
}
```

OUTPUT:



TASK 6.3:TWO STRINGS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static String twoStrings(String s1, String s2) {
        boolean[] seen = new boolean[26];
        for (char c : s1.toCharArray()) {
            seen[c - 'a'] = true;
        }
        for (char c : s2.toCharArray()) {
            if (seen[c - 'a']) {
                return "YES";
            }
        }
        return "NO";
    }
    public static void main(String[] args) throws Exception {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            String s1 = br.readLine().trim();
            String s2 = br.readLine().trim();


            System.out.println(twoStrings(s1, s2));
        }
    }
}
```

OUTPUT:

Expected Output

```
1  YES
2  NO
```

TASK 6.4:ROTATE STRING

PROGRAM:

```
class Solution {
    public boolean rotateString(String s, String goal) {

        if (s.length() != goal.length()) {
            return false;
        }

        return (s + s).contains(goal);
    }
}
```

OUTPUT:

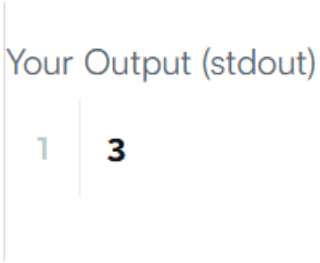


TASK 6.5:MARS EXPLORATION

PROGRAM:

```
import java.util.*;
public class Solution {
    public static int marsExploration(String s) {
        int count = 0;
        String message = "SOS";
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != message.charAt(i % 3)) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int result = marsExploration(s);
        System.out.println(result);
        sc.close();
    }
}
```

OUTPUT:



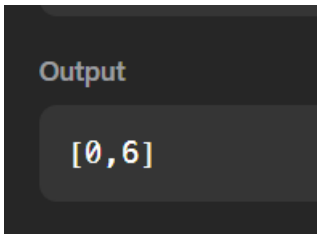
TASK 6.6: FIND ALL ANAGRAM

PROGRAM:

```
import java.util.*;
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        if (s.length() < p.length()) {
            return result;
        }
        int[] count = new int[26];
        for (char c : p.toCharArray()) {
            count[c - 'a']++;
        }
        int left = 0;
        int right = 0;
        int required = p.length();
        while (right < s.length()) {
            char c = s.charAt(right);
            if (count[c - 'a'] > 0) {
                required--;
            }
            count[c - 'a']--;
            right++;
            if (right - left == p.length()) {
                if (required == 0) {
                    result.add(left);
                }
                char leftChar = s.charAt(left);
                count[leftChar - 'a']++;
```

```
            if (count[leftChar - 'a'] > 0) {
                required++;
            }
            left++;
        }
    }
    return result;
  }
}
```

OUTPUT:



TASK 6.7:PALINDROME INDEX

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static int palindromeIndex(String s) {
        int left = 0;
        int right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                if (isPalindrome(s, left + 1, right)) {
                    return left;
                }
                if (isPalindrome(s, left, right - 1)) {
                    return right;
                }
                return -1;
            }
            left++;
            right--;
        }
        return -1;
    }
    private static boolean isPalindrome(String s, int left, int right) {
        while (left < right) {
            if (s.charAt(left) != s. charAt(right)) {
                return false;
            }
            left++;
            right--;
```

```
        }
        return true;
    }
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        int q = sc.nextInt();
        while (q-- > 0) {
            String s = sc.next();
            System.out.println(palindromeIndex(s));
        }
        sc.close();
    }
}
```

OUTPUT:

Expected Output

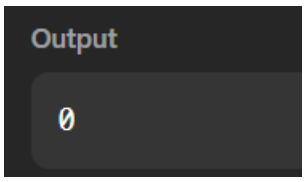
```
3
0
-1
```

TASK 6.8:FIND THE INDEX

PROGRAM:

```
class Solution {
    public int strStr(String haystack, String needle) {
        int n = haystack.length();
        int m = needle.length();
        if (m > n) {
            return -1;
        }
        for (int i = 0; i <= n - m; i++) {
            int j = 0;
            while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
                j++;
            }
            if (j == m) {
                return i;
            }
        }
        return -1;
    }
}
```

OUTPUT:



TASK 6.9:LONGEST PALINDROMIC SUBSTRING

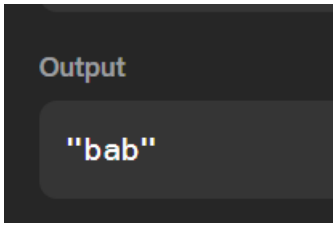
PROGRAM:

```
class Solution {
    public String longestPalindrome(String s) {
        int start = 0;
        int maxLength = 1;
        for (int i = 0; i < s.length(); i++) {
            int left = i;
            int right = i;
            while (left >= 0 && right < s.length()
                    && s.charAt(left) == s.charAt(right)) {
                if (right - left + 1 > maxLength) {
                    start = left;
                    maxLength = right - left + 1;
                }
                left--;
                right++;
            }

            left = i;
            right = i + 1;
            while (left >= 0 && right < s.length()
                    && s.charAt(left) == s.charAt(right)) {
                if (right - left + 1 > maxLength) {
                    start = left;
                    maxLength = right - left + 1;
                }
                left--;
                right++;
            }
```

```
        }
        return s.substring(start, start + maxLength);
    }
}
```

OUTPUT:



TASK 6.10:CIRCULAR PALINDROMES

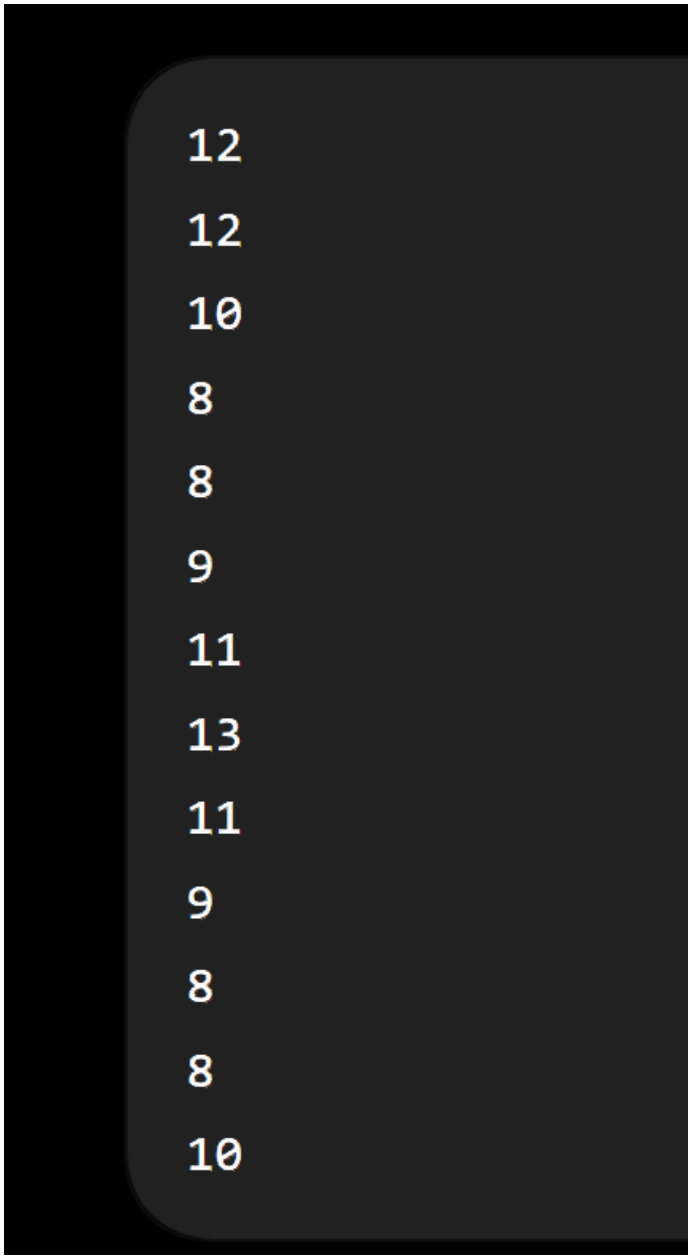
PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    static int[] manacher(String s) {
        int n = s.length();
        StringBuilder t = new StringBuilder();
        t.append('#');
        for (int i = 0; i < n; i++) {
            t.append(s.charAt(i));
            t.append('#');
        }
        int m = t.length();
        int[] p = new int[m];
        int center = 0;
        int right = 0;
        for (int i = 0; i < m; i++) {
            int mirror = 2 * center - i;
            if (i < right) {
                p[i] = Math.min(right - i, p[mirror]);
            }
            while (i + p[i] + 1 < m &&
                   i - p[i] - 1 >= 0 &&
                   t.charAt(i + p[i] + 1)
                   == t.charAt(i - p[i] - 1)) {
                p[i]++;
            }
            if (i + p[i] > right) {
                center = i;
```

```
            right = i + p[i];
        }
    }
    return p;
}
static int longestPalindrome(String s) {
    if (s.length() == 0) {
        return 0;
    }
    int[] p = manacher(s);
    int maximum = 0;
    for (int i = 0; i < p.length; i++) {
        if (p[i] > maximum) {
            maximum = p[i];
        }
    }
    return maximum;
}
public static List<Integer> circularPalindromes(String s) {
    int n = s.length();
    List<Integer> result = new ArrayList<Integer>();
    String current = s;
    for (int rotation = 0; rotation < n; rotation++) {
        int maximum = longestPalindrome(current);
        result.add(maximum);
        current = current.substring(1)
                + current.charAt(0);
    }
    return result;
}
```

```
    public static void main(String[] args) throws Exception {
        BufferedReader br =
                new BufferedReader(
                        new InputStreamReader(System.in)
                );
        int n = Integer.parseInt(br.readLine().trim());
        String s = br.readLine().trim();
        List<Integer> answer =
                circularPalindromes(s);
        StringBuilder output =
                new StringBuilder();
        for (int i = 0; i < answer.size(); i++) {
            output.append(answer.get(i));
            if (i < answer.size() - 1) {
                output.append('\n');
            }
        }
        System.out.print(output.toString());
    }
}
```

OUTPUT:


12
12
10
8
8
9
11
13
11
9
8
8
10

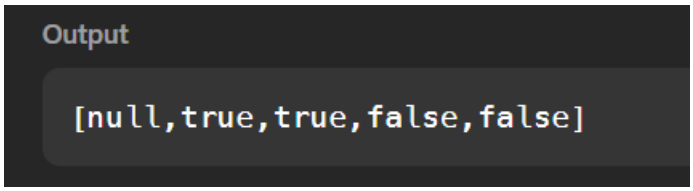
TASK 7.1: DESIGN PARKING SYSTEM

PROGRAM:

```
class ParkingSystem {
    int big;
    int medium;
    int small;
    public ParkingSystem(int big, int medium, int small) {
        this.big = big;
        this.medium = medium;
        this.small = small;
    }
    public boolean addCar(int carType) {
        if (carType == 1) {
            if (big > 0) {
                big--;
                return true;
            }
        }
        else if (carType == 2) {
            if (medium > 0) {
                medium--;
                return true;
            }
        }
        else if (carType == 3) {
            if (small > 0) {
                small--;
                return true;
            }
        }
```

```
        return false;
    }
}
```

OUTPUT:



TASK 7.2 : JAVA INSTANCES OF A KEYWORD

PROGRAM:

```
import java.util.*;
class Student{}
class Rockstar{}
class Hacker{}
public class InstanceOFTutorial{

    static String count(ArrayList mylist){
        int a = 0, b = 0, c = 0;

        for(int i = 0; i < mylist.size(); i++){
            Object element = mylist.get(i);

            if(element instanceof Student)
                a++;

            if(element instanceof Rockstar)
                b++;

            if(element instanceof Hacker)
                c++;
        }

        String ret = Integer.toString(a) + " "
                + Integer.toString(b) + " "
                + Integer.toString(c);

        return ret;
    }
```

```
    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        for(int i = 0; i < t; i++){
            String s = sc.next();

            if(s.equals("Student"))
                mylist.add(new Student());

            if(s.equals("Rockstar"))
                mylist.add(new Rockstar());

            if(s.equals("Hacker"))
                mylist.add(new Hacker());
        }

        System.out.println(count(mylist));
    }
}import java.util.*;

class Student{}
class Rockstar{}
class Hacker{}

public class InstanceOFTutorial{
```

```
static String count(ArrayList mylist){
    int a = 0, b = 0, c = 0;

    for(int i = 0; i < mylist.size(); i++){
        Object element = mylist.get(i);

        if(element instanceof Student)
            a++;

        if(element instanceof Rockstar)
            b++;

        if(element instanceof Hacker)
            c++;
    }

    String ret = Integer.toString(a) + " "
            + Integer.toString(b) + " "
            + Integer.toString(c);

    return ret;
}

public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);

    int t = sc.nextInt();

    for(int i = 0; i < t; i++){
```

```
            String s = sc.next();

            if(s.equals("Student"))
                mylist.add(new Student());

            if(s.equals("Rockstar"))
                mylist.add(new Rockstar());

            if(s.equals("Hacker"))
                mylist.add(new Hacker());
        }

        System.out.println(count(mylist));
    }
}
```

OUTPUT:

Expected Output

```
3 1 1
```

TASK 7.3 : DESIGN UNDERGROUND SYSTEM

PROGRAM:

```
import java.util.*;
class UndergroundSystem {
    HashMap<Integer, CheckIn> checkIns;
    HashMap<String, Route> routes;
    class CheckIn {
        String station;
        int time;
        CheckIn(String station, int time) {
            this.station = station;
            this.time = time;
        }
    }
    class Route {
        int totalTime;
        int count;
        Route(int totalTime, int count) {
            this.totalTime = totalTime;
            this.count = count;
        }
    }
    public UndergroundSystem() {
        checkIns = new HashMap<>();
        routes = new HashMap<>();
    }
    public void checkIn(int id, String stationName, int t) {
        checkIns.put(id, new CheckIn(stationName, t));
    }
```

```
    public void checkOut(int id, String stationName, int t) {
        CheckIn c = checkIns.get(id);
        String key = c.station + "#" + stationName;
        int travelTime = t - c.time;
        if (routes.containsKey(key)) {
            Route r = routes.get(key);
            r.totalTime += travelTime;
            r.count++;
        } else {
            routes.put(key, new Route(travelTime, 1));
        }
        checkIns.remove(id);
    }
    public double getAverageTime(String startStation, String endStation) {
        String key = startStation + "#" + endStation;
        Route r = routes.get(key);
        return (double) r.totalTime / r.count;
    }
}
```

OUTPUT:

Output

```
[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,nul
l,12.00000]
```

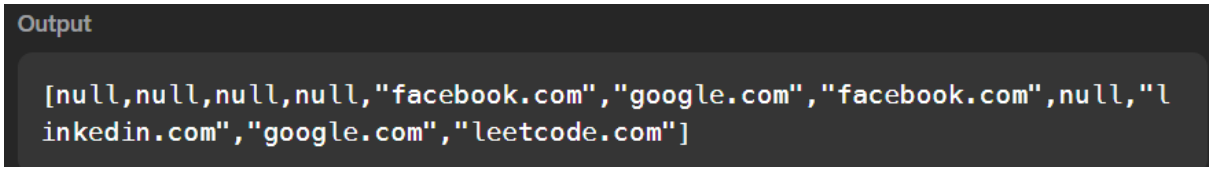
TASK 7.4 : DESIGN BROWSER HISTORY

PROGRAM:

```
class BrowserHistory {
    class Node {
        String url;
        Node prev;
        Node next;
        Node(String url) {
            this.url = url;
        }
    }
    Node current;
    public BrowserHistory(String homepage) {
        current = new Node(homepage);
    }
    public void visit(String url) {
        Node newNode = new Node(url);
        current.next = newNode;
        newNode.prev = current;
        current = newNode;
    }
    public String back(int steps) {
        while (steps > 0 && current.prev != null) {
            current = current.prev;
            steps--;
        }
        return current.url;
    }
    public String forward(int steps) {
        while (steps > 0 && current.next != null) {
```

```
            current = current.next;
            steps--;
        }
        return current.url;
    }
}
```

OUTPUT:



TASK 7.5 : HASHSET

PROGRAM:

```
class MyHashSet {
    boolean[] set;
    public MyHashSet() {
        set = new boolean[1000001];
    }
    public void add(int key) {
        set[key] = true;
    }
    public void remove(int key) {
        set[key] = false;
    }
    public boolean contains(int key) {
        return set[key];
    }
}
```

OUTPUT:

Output

```
[null,null,null,null,null,false,null,null,null,null,false,null,null,nul
l,null,false,null,null,null,null,false]
```

TASK 7.6 : GRADING STUDENTS

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;
class Result {
    public static List<Integer> gradingStudents(List<Integer> grades) {
        for (int i = 0; i < grades.size(); i++) {
            int grade = grades.get(i);
            if (grade >= 38) {
                int next = ((grade / 5) + 1) * 5;
                if (next - grade < 3) {
                    grades.set(i, next);
                }
            }
        }
        return grades;
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
                new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter =
                new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
        int gradesCount =
                Integer.parseInt(bufferedReader.readLine().trim());
        List<Integer> grades = IntStream.range(0, gradesCount)
                .mapToObj(i -> {
```

```
                try {
                    return bufferedReader.readLine().replaceAll("\\s+$", "");
                } catch (IOException ex) {
                    throw new RuntimeException(ex);
                }
            })
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> result = Result.gradingStudents(grades);
        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(Collectors.joining("\n"))
                + "\n"
        );
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Your Output (stdout)

```
75
67
40
33
```

TASK 7.7 : JAVA INHERITANCE 1

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
class Animal {
    void walk() {
        System.out.println("I am walking");
    }
}
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
    void sing() {
        System.out.println("I am singing");
    }
}
public class Solution {
    public static void main(String[] args) {
        Bird bird = new Bird();
        bird.walk();
        bird.fly();
        bird.sing();
    }
}
```

OUTPUT:

Your Output (stdout)

```
I am walking
I am flying
I am singing
```

TASK 7. 8: JAVA INHERITANCE 2

PROGRAM:

```
import java.io.*;
import java.util.*;
class Arithmetic {
    int add(int a, int b) {
        return a + b;
    }
}
class Adder extends Arithmetic {
}
public class Solution {
    public static void main(String[] args) {
        Adder adder = new Adder();
        System.out.println("My superclass is: "
                + adder.getClass().getSuperclass().getName());
        System.out.println(adder.add(42, 13) + " "
                + adder.add(10, 10) + " "
                + adder.add(5, 15));
    }
}
```

OUTPUT:

Expected Output

```
My superclass is: Arithmetic
42 13 20
```

TASK 7.9 : JAVA GENERICS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3};
        String[] stringArray = {"Hello", "World"};
        printArray(intArray);
        printArray(stringArray);
    }
}
```

OUTPUT:



TASK 7.10: THRONE INHERITANCE

PROGRAM:

```
import java.util.*;
class ThroneInheritance {
    String king;
    Map<String, List<String>> children;
    Set<String> dead;
    public ThroneInheritance(String kingName) {
        king = kingName;
        children = new HashMap<>();
        dead = new HashSet<>();
    }
    public void birth(String parentName, String childName) {
        children.putIfAbsent(parentName, new ArrayList<>());
        children.get(parentName).add(childName);
    }
    public void death(String name) {
        dead.add(name);
    }
    public List<String> getInheritanceOrder() {
        List<String> result = new ArrayList<>();
        dfs(king, result);
        return result;
    }
    private void dfs(String person, List<String> result) {
        if (!dead.contains(person)) {
            result.add(person);
        }
        if (children.containsKey(person)) {
            for (String child : children.get(person)) {
```

```
            dfs(child, result);
        }
    }
  }
}
```

OUTPUT:

Output

```
[null,null,null,null,null,null,null,["king","andy","matthew","bob","a
x","asha","catherine"],null,["king","andy","matthew","alex","asha","cat
herine"]]
```

WEEK 8

TASK 8. 1 :JAVA METHOD OVERRIDING

PROGRAM:

```
import java.io.*;
import java.util.*;
class Sports{
   String getName(){
      return "Generic Sports";
   }
   void getNumberOfTeamMembers(){
      System.out.println("Each team has n players in " + getName());
   }
}
class Soccer extends Sports{
   @Override
   String getName(){
      return "Soccer Class";
   }
   @Override
   void getNumberOfTeamMembers(){
      System.out.println("Each team has 11 players in " + getName());
   }
}
public class Solution{
   public static void main(String[] args){
      Sports c1 = new Sports();
      Soccer c2 = new Soccer();
      System.out.println(c1.getName());
      c1.getNumberOfTeamMembers();
```

```
        System.out.println(c2.getName());
        c2.getNumberOfTeamMembers();
    }
}
```

OUTPUT:

Expected Output

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

TASK 8.2: JAVA METHOD OVERRIDING 2 (SUPER KEYWORD)

PROGRAM:

```
import java.io.*;
import java.util.*;
class BiCycle{
    String define_me(){
        return "a vehicle with pedals.";
    }
}
class MotorCycle extends BiCycle{
    String define_me(){
        return "a cycle with an engine.";
    }
    MotorCycle(){
        System.out.println("Hello I am a motorcycle, I am " + define_me());
        String temp = super.define_me();
        System.out.println("My ancestor is a cycle who is " + temp);
    }
}
public class Solution{
    public static void main(String[] args){
        MotorCycle M = new MotorCycle();
    }
}
```

OUTPUT:

Expected Output Download

```
1 Hello I am a motorcycle, I am a cycle with an engine.
2 My ancestor is a cycle who is a vehicle with pedals.
```

TASK 8.3: JAVA SINGLETONE PATTERN

PROGRAM:

TASK 8.4:

TASK 8 .5 : JAVA ITERATOR

PROGRAM:

```
import java.util.*;
public class Solution {
    static void func(ArrayList mylist) {
        Iterator it = mylist.iterator();
        while (it.hasNext()) {
            Object element = it.next();
            if (element instanceof String) {
                break;
            }
        }
        while (it.hasNext()) {
            Object element = it.next();
            System.out.println((String) element);
        }
    }
    public static void main(String[] args) {
        ArrayList mylist = new ArrayList();
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            mylist.add(sc.nextInt());
        }
        mylist.add("###");
        int m = sc.nextInt();
        for (int i = 0; i < m; i++) {
            mylist.add(sc.next());
        }
```

```
        func(mylist);
    }
}
```

OUTPUT:

Your Output (stdout)

```
hello
java
```

TASK 8.6: JAVA INTERFACE

PROGRAM:

```
import java.io.*;
import java.util.*;
interface AdvancedArithmetic {
   int divisor_sum(int n);
}
class MyCalculator implements AdvancedArithmetic {
   public int divisor_sum(int n) {
      int sum = 0;
      for (int i = 1; i <= n; i++) {
         if (n % i == 0) {
            sum += i;
         }
      }
      return sum;
   }
}
public class Solution {
   public static void main(String[] args) {
      MyCalculator myCalculator = new MyCalculator();
      System.out.println("I implemented: " +
myCalculator.getClass().getInterfaces()[0].getSimpleName());
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      System.out.println(myCalculator.divisor_sum(n));
      sc.close();
   }
}
```

OUTPUT:

Your Output (stdout)

```
1  I implemented:
   AdvancedArithmetic
2  12
```

TASK 8.7:

TASK 8.8 : JAVA FACTORY

PROGRAM:

```
import java.util.*;
import java.security.*;
interface Food {
   public String getType();
}
class Pizza implements Food {
   public String getType() {
      return "Someone ordered Fast Food!";
   }
}
class Cake implements Food {
   public String getType() {
      return "Someone ordered a Dessert!";
   }
}
class FoodFactory {
   public Food getFood(String order) {
      if (order.equals("pizza")) {
         return new Pizza();
      } else {
         return new Cake();
      }
   }
}
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      FoodFactory foodFactory = new FoodFactory();
```

```
        Food food = foodFactory.getFood(sc.nextLine());
        System.out.println("The factory returned " + food.getClass());
        System.out.println(food.getType());
    }
}
```

OUTPUT:

Your Output (stdout)

```
1  The factory returned class Cake
2  Someone ordered a Dessert!
```

TASK 8.9: JAVA EXCEPTIONAL HANDLING

PROGRAM:

```
import java.io.*;
import java.util.*;
class MyCalculator {
    public long power(int n, int p) throws Exception {
        if (n < 0 || p < 0) {
            throw new Exception("n or p should not be negative.");
        }
        if (n == 0 && p == 0) {
            throw new Exception("n and p should not be zero.");
        }
        return (long) Math.pow(n, p);
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        MyCalculator myCalculator = new MyCalculator();
        while (in.hasNextInt()) {
            int n = in.nextInt();
            int p = in.nextInt();
            try {
                System.out.println(myCalculator.power(n, p));
            } catch (Exception e) {
                System.out.println(e);
            }
        }
        in.close();
    }
```

}

OUTPUT:

Your Output (stdout)

```
243
16
java.lang.Exception: n and p should not be zero.
java.lang.Exception: n or p should not be negative.
java.lang.Exception: n or p should not be negative.
```

TASK 8.10: JAVA EXCEPTIONAL HANDLING (Try – catch)

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            int a = sc.nextInt();
            int b = sc.nextInt();
            System.out.println(a / b);
        } catch (InputMismatchException e) {
            System.out.println("java.util.InputMismatchException");
        } catch (ArithmeticException e) {
            System.out.println(e);
        }
        sc.close();
    }
}
```

OUTPUT:

